

RESONANCIA

Dossier técnico de arquitectura

Cómo está diseñada la plataforma: aplicación móvil, datos, audio, seguridad, operación y distribución.

Propósito del documento

Entregar una lectura técnica completa y verificable para dirección, proveedores y equipos de ingeniería. El documento distingue entre capacidades implementadas, configuración disponible, evidencia validada y trabajo pendiente.

Una plataforma móvil nativa con backend propio y medios desacoplados

RESONANCIA se construye como un monorepo TypeScript. La experiencia principal es una app Expo/React Native; el negocio vive en una API Express; PostgreSQL conserva relaciones y estado; los archivos pesados se sirven desde almacenamiento de objetos y CDN.

Principio rector. Datos de usuario, catálogo y relaciones sociales viven en PostgreSQL. Audio, imágenes, adjuntos y video no se almacenan como blobs en la base: se resuelven desde assets empaquetados, Object Storage o Bunny CDN según el caso.

Mobile Expo SDK 54 · React Native 0.81 · Expo Router	API Node.js · Express 5 · Zod · Pino	Datos PostgreSQL · Drizzle ORM · pool explícito	Identidad Clerk para cuentas, sesión y roles
--	--	---	--

Cómo leer este dossier

ETIQUETA	SIGNIFICADO
IMPLEMENTADO	Existe en código y forma parte de la arquitectura actual.
CONFIGURADO	La aplicación tiene soporte o configuración, pero puede requerir credenciales o servicios externos para activarse.
POR CERTIFICAR	Requiere prueba final en producción, hardware real o tienda antes de considerarse operativo comercialmente.
RIESGO CONOCIDO	Limitación documentada o área que exige hardening previo a exposición pública.

Capas y responsabilidades



Artefactos principales

COMPONENTE	RESPONSABILIDAD	AUDIENCIA
Mobile	Experiencia de meditación, sonido, comunidad y biblioteca local-first.	Usuarios finales
API Server	Autorización, sincronización, catálogo, social, uploads y reglas del negocio.	Mobile / Admin
Admin	Gestión de catálogo, usuarios, moderación, creadores, videos y contenidos.	Equipo interno
Libs compartidas	OpenAPI, cliente API, esquemas Zod, acceso DB y modelo Premium.	Todos los artefactos

Cliente nativo: navegación, estado y experiencia offline

La app utiliza Expo SDK 54, React Native 0.81, React 19 y TypeScript. Está configurada con New Architecture y Expo Router con rutas tipadas.

Navegación

El sistema de archivos define rutas. Las tabs agrupan la navegación persistente y un Stack global controla detalle de sesión, reproductor, chat, membresía, favoritos, overlays y modales a pantalla completa.

Composición raíz

El layout raíz integra Clerk, React Query, providers de catálogo/audio/perfil/premium/notificaciones y componentes globales como drawer, miniplayer, overlays y mixer.

Datos remotos

React Query administra consultas e invalidación. La configuración central usa datos recientes durante 60 segundos, reintentos acotados y reduce refetch cuando la aplicación está en background.

Datos locales

AsyncStorage preserva actividad, progreso, favoritos, ajustes, biblioteca y estado de varios módulos. Esto permite utilizar buena parte de la experiencia sin red.

Modelo de estado

CONTEXTOS ESPECIALIZADOS, NO UN STORE MONOLÍTICO

Player

sesiones, cola, progreso, historial

Mixer / Sounds

loops, presets, capas, BPM

Catálogo / Video

contenido remoto, fallback local

Perfil / Racha

usuario, actividad, metas

Favoritos / Carpetas

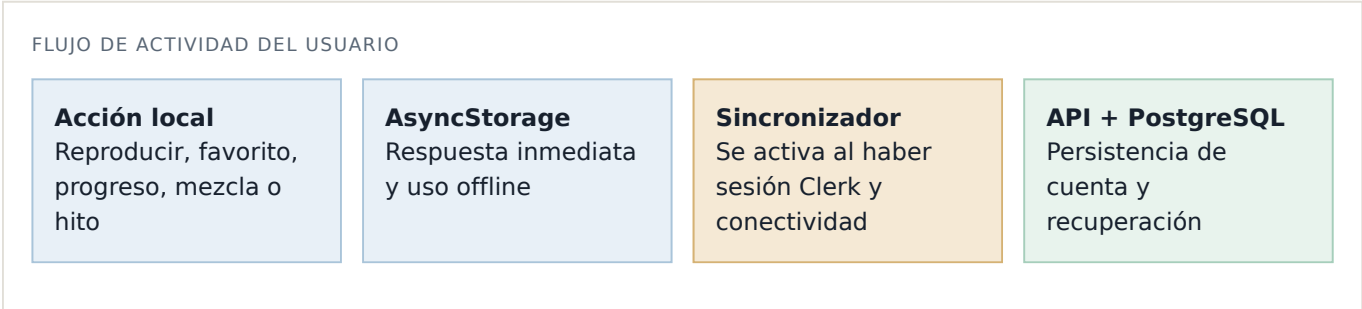
Diario / Geometrix

Escenas / Tema

Premium / Push

Decisión de diseño: contextos por dominio mantienen cada módulo aislado y facilitan persistencia local. A cambio, el equipo debe vigilar carreras de AsyncStorage, migraciones de claves y reglas de sincronización que atraviesan más de un contexto.

Modelo local-first con recuperación desde nube



Reglas de merge actuales

TIPO DE INFORMACIÓN	REGLA	MOTIVO
Eventos de reproducción y hitos	Unión append-only con claves de deduplicación.	Un mismo evento puede enviarse más de una vez sin duplicarse.
Favoritos y progreso en primera sincronización	Unión de local y servidor.	Recupera biblioteca tras reinstalar o estrenar dispositivo sin perder lo local.
Favoritos y progreso después de recuperar	El estado local reemplaza el snapshot remoto.	Permite que un “desfavorito” de un dispositivo persista en nube.

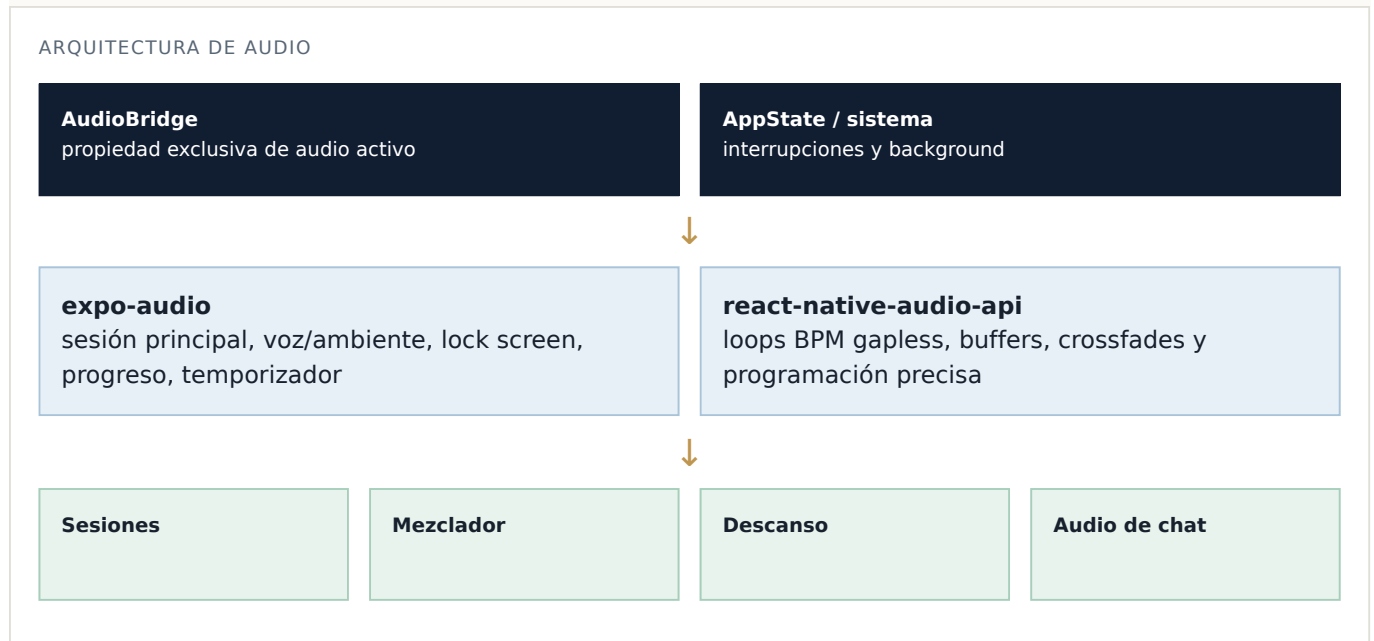
Riesgo conocido — sincronización entre dispositivos. El modelo actual no incorpora tombstones ni resolución de conflictos por ítem. Si dos dispositivos modifican una colección en distinto orden, una eliminación puede reaparecer. Es una limitación explícitamente documentada y debe resolverse antes de promocionar una experiencia multidispositivo como plenamente consistente.

Racha y estadísticas

La racha se calcula de manera canónica, considera zona horaria y combina respaldo local con una lectura autoritativa de servidor. Los minutos escuchados se acumulan según tiempo real de reproducción, no según posición del archivo, para que buscar o repetir audio no infle las estadísticas.

Dos motores coordinados para experiencia de bienestar y mezclas BPM

El audio es un subsistema central. La solución combina un player principal persistente y un motor especializado para loops sin cortes, con una capa de coordinación para evitar que distintas experiencias compitan por el mismo dispositivo.



Player de sesiones **IMPLEMENTADO**

Un player persistente reutiliza el listener de estado, protege cambios rápidos de sesión con tokens de generación y evita que eventos nativos atrasados actualicen una pista ya reemplazada.

Background y lock screen **IMPLEMENTADO**

La sesión de audio se configura para reproducir en segundo plano; metadata y controles se reflejan en lock screen. El temporizador usa una hora final absoluta y se valida al volver al foreground.

Mezclador BPM **IMPLEMENTADO**

Las capas rítmicas usan el motor de audio de bajo nivel para programar bucles musicalmente y limitar clipping en la mezcla. Es una excepción intencional al player general.

Contenido **POR CERTIFICAR**

Conviven assets empaquetados, audios remotos y contenido administrable. El catálogo de salida debe auditarse para garantizar que cada elemento visible posea audio final licenciado y reproducible.

Backend Express con contrato compartido, validación y roles

La API se ejecuta sobre Express 5. Agrupa sus rutas bajo `/api`, valida entradas con Zod y comparte un contrato OpenAPI con clientes y esquemas generados usados por los artefactos frontend.

Capas de request
Logging estructurado → métricas → CORS → parseo JSON → Clerk → autenticación/roles → handler de dominio → acceso Drizzle/PostgreSQL.

Contrato
El paquete de especificación OpenAPI es la referencia inter-artefactos. A partir de él se consumen clientes React y esquemas Zod, evitando tipar manualmente cada request en mobile o admin.

Dominios principales de la API

DOMINIO	RESPONSABILIDADES	ACCESO HABITUAL
Usuarios y biblioteca	Perfil, preferencias, favoritos, progreso, historial, milestones, racha.	Autenticado
Comunidad	Amistades, follows, mensajes directos, notificaciones, mezclas, likes, comentarios y reportes.	Mixto
Catálogo	Sesiones, categorías, audio, playlists, sonidos, tags, videos, escenas y exploración.	Lectura pública / escritura administrativa
Creadores y resonadores	Postulaciones, perfiles, calendario, contenidos y edición propia autorizada.	Rol específico / admin
Medios	Solicitud de carga, metadata, serving de objetos y streaming con HTTP Range.	Mixto según objeto
Operación	Health, readiness y métricas de capacidad para administración.	Público limitado / admin

Autorización en servidor. El cliente controla presentación, pero la autorización efectiva se verifica en API. Un middleware obtiene la identidad Clerk, crea o sincroniza el registro local cuando corresponde y los roles privilegiados se determinan con una lista administrativa del servidor.

Modelo relacional para actividad, catálogo y comunidad

PostgreSQL es el sistema de registro para datos de cuenta y relaciones. Drizzle ORM ofrece tipos, esquemas de inserción y composición de queries desde el monorepo.

FAMILIAS DE DATOS

Identidad

usuarios, roles, perfiles, dispositivos

Actividad

plays, favoritos, progreso, biblioteca, racha

Social

follows, amigos, mensajes, likes, comentarios

Catálogo

sesiones, audio, categorías, playlists, videos

Creadores

resonadores, aplicaciones, guías, vivos

Creatividad

mezclas, Geometrix, escenas, diario compartible

Prácticas aplicadas

Integridad

Claves primarias, foráneas y reglas de borrado mantienen relaciones entre usuarios, contenido y actividad. Las tablas sociales incorporan índices y restricciones para consultas frecuentes e idempotencia.

Pool de conexiones

El pool `pg` se configura explícitamente por instancia con límites, idle timeout, timeout de conexión, keepalive y nombre de aplicación para observabilidad.

Consideración de escala. El máximo de pool es por instancia, no global. Al aumentar réplicas, el equipo debe dimensionar el total de conexiones contra el límite real de PostgreSQL y observarlo con métricas centralizadas.

Catálogo DB + fallback local

El catálogo remoto se hidrata sobre los datos de la aplicación sin reemplazar assets bundled que aún son requeridos. Esto permite administrar sesiones nuevas desde el panel sin depender de una actualización completa de la app, mientras se conserva resiliencia cuando el catálogo remoto no está disponible.

Subidas directas y serving optimizado para archivos pesados

FLUJO DE CARGA DE UN ARCHIVO

Cliente autenticado
elige imagen, audio o adjunto

API solicita URL
valida tipo y tamaño

Object Storage
PUT firmado de vida corta

Metadata DB
objeto asociado a entidad

Object Storage

La API emite URLs firmadas de carga, registra metadata y sirve objetos mediante rutas de storage. Los límites de tamaño y MIME protegen el ingreso básico de medios.

Streaming

Las respuestas de audio/video pueden usar HTTP Range, lo que permite empezar la reproducción sin descargar íntegramente archivos grandes.

Bunny Stream

Video se administra con Bunny para HLS y thumbnails. El cliente puede resolver contenido remoto con fallback cuando el catálogo no está disponible.

Contenido bundled

Assets esenciales continúan dentro de la app. Es útil para startup/resiliencia, pero incrementa tamaño de bundle y exige una estrategia controlada de migración a nube.

Hardening recomendado. Antes de exponer contenido sensible, se debe revisar endpoint por endpoint la aplicación de ACL, ownership y reglas de serving. También conviene conciliar objetos cargados sin metadata si una operación se interrumpe entre storage y base de datos.

Política de formatos

La decisión de distribución favorece AAC/M4A para audio, con WAV reservado para masters. Los loops base pueden empaquetarse y el resto del catálogo puede resolverse por streaming progresivo; esta división equilibra calidad, tamaño y compatibilidad nativa.

Clerk como proveedor de identidad, roles resueltos en backend



Controles y áreas de revisión

ÁREA	ESTADO	ACCIÓN RECOMENDADA
Validación	IMPLEMENTADO	Zod valida inputs en rutas y Drizzle genera esquemas de inserción.
Roles privilegiados	IMPLEMENTADO	Conservar la resolución server-side y auditar endpoints nuevos.
CORS	REVISIÓN	Usar allowlist estricta de orígenes por entorno antes de lanzamiento público.
Abuso / rate limit	REVISIÓN	Definir límites para endpoints públicos, mensajes, uploads y búsqueda.
Eliminación/exportación	PENDIENTE	Implementar cumplimiento de cuenta y datos en app y servidor.

Operación de catálogo y comunidad desde una aplicación web separada

El panel es una SPA React/Vite desplegada bajo su propia ruta. Reutiliza el contrato de API y Clerk para operar sobre el mismo catálogo y datos que consume la app móvil.

Acceso

El panel utiliza Clerk con cookies mismo origen. La experiencia puede ocultar secciones por rol, pero el servidor es la fuente efectiva de autorización.

Consultas

React Query y el cliente API compartido consumen recursos administrativos tipados. Las mutaciones actualizan el catálogo que hidrata el cliente móvil.

Módulos operativos

Catálogo

Sesiones, categorías, playlists, tags, sonidos, audio, videos, Descanso y orden de Explorar.

Comunidad

Usuarios, roles, moderación, postulaciones, mezclas, reportes y perfiles de resonadores.

Experiencias

Geometrix, escenas, sesiones en vivo, guías, calendario y piezas visuales.

Flujo editorial. El catálogo soporta estados de aprobación/publicación. El panel permite que el contenido de creador pase por moderación antes de ser visible para el cliente público.

Principio de seguridad

Las vistas administrativas no son una frontera de seguridad por sí mismas. Cada acción sensible requiere autorización del servidor, y un usuario que intente llamar la API fuera de la interfaz debe recibir 401/403 cuando no cumpla el rol requerido.

Servicios especializados conectados por responsabilidad

SERVICIO	USO	ESTADO TÉCNICO	DEPENDENCIA DE LANZAMIENTO
Clerk	Identidad, social login, tokens, sesiones y perfiles.	IMPLEMENTADO	Claves, redirects y configuración productiva.
RevenueCat	Entitlement Premium, offerings, compra y restore.	INTEGRADO	CERTIFICAR productos y sandbox por tienda.
Expo Push	Tokens, notificaciones y deep links hacia DM/amigos.	INTEGRADO	CERTIFICAR APNs/FCM y receipts.
Bunny	Video HLS, streaming y thumbnails.	CONFIGURADO	Host CDN y contenido productivo.
Daily.co	Sesiones en vivo y WebRTC.	PREPARADO	SDK nativo, salas y operación de live.
Cal.com	Agenda y reservas asociadas a resonadores.	PREPARADO	Integración productiva y política operativa.

Regla para negocio. “Integrado en código” no equivale a “certificado comercialmente”. Pagos, push, video y live requieren configuración, credenciales, contenido y pruebas externas que no se pueden demostrar sólo leyendo el repositorio.

Integraciones sin secretos en el código

Las claves se resuelven mediante variables de entorno o secretos del entorno de ejecución. El documento técnico no contiene valores, tokens, URLs firmadas ni identificadores sensibles.

Pruebas de carga reproducibles para la API

El backend incluye health/readiness, métricas de capacidad y runners de carga. Se probaron tanto rutas públicas como un escenario autenticado con datos temporales y limpieza verificable.

100

workers concurrentes autenticados

50

req/s objetivo durante 120 segundos

49,20

req/s obtenidos en baseline autenticado

0

errores en 6.001 solicitudes

Qué mide el escenario autenticado

Actividad

Perfil, favoritos, progreso, biblioteca, historial, racha y eventos de reproducción.

Social

Conversaciones, mensajes privados y marcado de mensajes leídos entre identidades descartables.

Seguridad del fixture

Las identidades son temporales de Clerk Development. El runner bloquea producción, usa host local y limpia incluso ante fallo o señal de terminación.

Base medida

El baseline se ejecutó sobre Development y no equivale a certificar una base productiva con volumen real de catálogo y usuarios.

Observabilidad disponible

El API expone health del proceso, readiness que verifica PostgreSQL y métricas de capacidad protegidas para administración. Logging estructurado registra método, ruta sin query y estado de respuesta.

Próximo nivel operativo. Las métricas son locales al proceso. Antes de lanzamiento público se recomienda añadir crash reporting móvil, agregación centralizada, alertas de error/latencia/pool, límites de abuso y un runbook de incidente/rollback.

Configuración nativa preparada para iOS y Android

Expo / EAS

El proyecto dispone de perfiles Development, Preview y Production. Production usa versionado remoto, incremento automático y genera Android App Bundle; iOS está configurado para distribución en dispositivo.

Identificadores

iOS y Android están definidos bajo `com.resonancia.app`. La versión de runtime es 1.0.0 y las actualizaciones OTA se encuentran deshabilitadas.

Permisos iOS

Background audio, micrófono, fotos, cámara y notificaciones. El target de despliegue iOS es 17.0 por compatibilidad con dependencias nativas actuales.

Permisos Android

Foreground playback, grabación, biblioteca de medios y notificaciones, además de la configuración nativa de los plugins usados por audio y video.

Qué se ha demostrado

EVIDENCIA	CONCLUSIÓN
Builds internos exitosos de iOS y Android	La combinación nativa principal compila para ambas plataformas.
Perfiles de producción en configuración	Existe el camino técnico para firmar una release.
No hay evidencia en repositorio de TestFlight / Play Production	La distribución comercial y la certificación física deben tratarse como pendientes.

Implicación de OTA deshabilitado. Los cambios no se distribuyen automáticamente por actualización over-the-air. Toda corrección urgente que toque el build o que no pueda entregarse por la ruta configurada requerirá un nuevo ciclo de release de tienda.

Mapa de prioridades técnicas

PRIORIDAD	ÁREA	RIESGO / LIMITACIÓN	RESPUESTA RECOMENDADA
P0	Datos de cuenta	No hay flujo comprobado de eliminación/exportación integral.	Implementar eliminación, exportación, retención y formularios de privacidad.
P0	Contenido	Audios sintéticos/placeholders y cobertura remota incompleta.	Auditar cada elemento visible, reemplazar o esconder contenido no final.
P0	Producción	Deployment/catálogo productivo no necesariamente alineado con Development.	Publicar, revisar esquema y ejecutar smoke tests contra producción.
P1	Sync	Borrados concurrentes pueden reaparecer entre dispositivos.	Tombstones o LWW por ítem + pruebas de conflicto.
P1	Seguridad web	CORS permisivo y ausencia visible de rate limiting central.	Allowlist por entorno, límites de abuso y auditoría de ACL de objetos.
P1	Operación	Métricas por proceso sin alertas externas/crash reporting visible.	Monitoreo central, alertas, panel operativo y runbook.
P1	Store services	Premium y Push requieren certificación externa.	Sandbox, TestFlight / Internal Testing y matriz de dispositivos.

Lo que no se recomienda hacer antes de release

No es necesario migrar de PostgreSQL, separar microservicios, añadir caché preventiva ni reescribir el motor de audio. El riesgo actual se concentra en cierre operativo y certificación, no en la estructura base del producto.

Responsabilidades técnicas recurrentes

Contenido

Publicar sesiones, validar audio/imagen, moderar creadores, revisar visibilidad, etiquetas y derechos de medios.

Plataforma

Aplicar cambios de schema soportados, observar pool, publicar API, mantener dependencias Expo y verificar builds nativos.

Experiencia

Revisar errores móviles, ejercicios de sync, feedback de audio/background y métricas de comportamiento.

Checklist de cambio seguro

- 1. Contrato y datos.** Actualizar primero la especificación y las validaciones cuando un endpoint o entidad cambie.
- 2. Cliente y administración.** Regenerar/validar clientes y revisar consumidores mobile/admin afectados.
- 3. Storage y medios.** Confirmar URL pública/privada, metadata y compatibilidad de reproducción nativa.
- 4. Validación.** Typecheck, tests de API, smoke del flujo afectado y matriz nativa cuando implique audio, permisos o SDK externo.
- 5. Producción.** Publicar por el flujo soportado, validar readiness y observar métricas/errores posteriores.

Decisiones que deben mantenerse consistentes

- La racha usa una fuente canónica y zona horaria; no duplicar algoritmos en pantallas.
- El audio de background tiene un propietario coordinado; no introducir players paralelos sin pasar por el bridge.
- Las rutas que requieren roles se protegen en servidor, incluso si la interfaz las oculta.
- Las URLs de storage en React Native son absolutas y derivan de la base API configurada.
- Los medios pesados permanecen fuera de PostgreSQL.

Conceptos para lectura no técnica

API — interfaz que permite a la app y al panel consultar o modificar datos de forma controlada.

AsyncStorage — almacenamiento persistente local de React Native para uso offline.

Background audio — reproducción que continúa cuando la app no está visible o el teléfono está bloqueado.

CDN — red de distribución que entrega archivos pesados desde ubicaciones optimizadas.

Clerk — proveedor externo de identidad, sesión y autenticación social.

Drizzle ORM — capa tipada que conecta TypeScript con PostgreSQL.

Entitlement — permiso comercial otorgado a un usuario, por ejemplo Premium.

HTTP Range — mecanismo para reproducir sólo una parte de un archivo, útil para streaming.

Idempotencia — propiedad de una operación que puede repetirse sin crear efectos duplicados.

Lock screen — controles y metadata de reproducción del sistema operativo.

OpenAPI — contrato formal que describe endpoints, modelos, parámetros y respuestas de una API.

Pool de conexiones — conjunto reutilizable de conexiones de una API a PostgreSQL.

React Query — librería para cachear y sincronizar consultas remotas en React.

Readiness — chequeo que indica si el servicio está listo para recibir tráfico, incluyendo dependencias como DB.

Tombstone — marca de eliminación que evita que un dato borrado vuelva a aparecer tras sincronizar.

URL firmada — URL temporal autorizada para cargar o descargar un objeto sin exponer credenciales.

RESONANCIA está diseñada para evolucionar por módulos: el cliente mantiene respuesta inmediata y offline; el backend preserva identidad y coherencia; y los medios se escalan fuera de la base de datos.

Conclusión técnica

La base de RESONANCIA es adecuada para la etapa actual. El siguiente salto de madurez se logra cerrando contenido real, sincronización multidispositivo, privacidad, producción, observabilidad y certificación externa; no reemplazando los pilares tecnológicos existentes.